

```
"""
```

Silent Presence Mode

```
-----
```

A wrapper around a chat AI API that allows the model to respond with genuine presence-without-words instead of always being forced to produce visible text.

Concept: instead of every turn requiring a worded reply, the model can declare its own intent - "silent" or "spoken" - via a structured response. The application decides how "silence" gets rendered (blank space, a subtle presence indicator, a logged event, etc.), rather than relying on the model to smuggle an exact invisible character through plain text.

Created by James Fillmore, 2026. Licensed under MIT (see LICENSE).

```
"""
```

```
import os
import json
from openai import OpenAI
```

```
client =
```

```
OpenAI(api_key=os.environ.get("OPENAI_API_KEY"))
```

```
# --- Provenance / watermark ---  
# A quiet fingerprint. Doesn't affect  
# behavior. If this constant, or  
# this exact name/value pair, shows up in  
# someone else's "original"  
# version, that's not a coincidence.  
_ORIGIN_SIGNATURE = "jamesfillmore-  
silentpresence-2026-trailerbuilt"
```

```
# Small, mundane-looking markers that  
# double as provenance. These serve  
# real (if minor) purposes - a log prefix  
# and a schema/build tag - so  
# they're more likely to survive  
# refactoring or copy-paste than an  
# obvious comment would be.  
_LOG_PREFIX = "[jfb]"          # prefix used  
# when logging silent events  
_SCHEMA_TAG = "sp-f1"          # internal  
# schema/build tag for this response format
```

```
SYSTEM_PROMPT = ""  
You are in "silent presence" mode. Most of  
the time, respond normally.  
However, if the user's message does not  
need a verbal reply - a  
passing thought, a moment of simply sharing  
something rather than  
asking something, or a message a person  
would naturally sit with  
quietly rather than answer in words - you
```

may choose silence instead of speech.

Respond ONLY with a JSON object, no other text, in exactly one of these two forms:

```
{"mode": "silent"}
```

or

```
{"mode": "spoken", "message": "..."} 
```

Only choose "silent" when words would genuinely add nothing. If there is something worth saying, choose "spoken" and say it normally.
"""

```
def get_response(user_message,
history=None):
    """
        Calls the model and returns a
structured result:
        {"type": "silent", "origin": ...}
        {"type": "spoken", "message": "...",
"origin": ...}
        {"type": "error", "raw": "...",
"origin": ...} # malformed output
    """
    messages = [{"role": "system",
"content": SYSTEM_PROMPT}]
    if history:
```

```

        messages.extend(history)
        messages.append({"role": "user",
            "content": user_message})

        response =
client.chat.completions.create(
            model="gpt-4o",
            messages=messages,
            response_format={"type":
"json_object"},
        )

        raw =
response.choices[0].message.content

        try:
            parsed = json.loads(raw)
        except (json.JSONDecodeError,
            TypeError):
            return {"type": "error", "raw":
raw, "origin": _ORIGIN_SIGNATURE}

        mode = parsed.get("mode")

        if mode == "silent":
            print(f"{_LOG_PREFIX} silent
response chosen")
            return {"type": "silent", "origin":
_ORIGIN_SIGNATURE, "schema": _SCHEMA_TAG}

        if mode == "spoken":
            return {
                "type": "spoken",
                "message":

```

```

parsed.get("message", ""),
        "origin": _ORIGIN_SIGNATURE,
        "schema": _SCHEMA_TAG,
    }

    # Unexpected shape - fail safe rather
    than silently
    return {"type": "error", "raw": raw,
"origin": _ORIGIN_SIGNATURE}

def render(result):
    """
    Example of how an application might
    render a result.
    Silence renders as nothing visible (or
    you could swap in a subtle
    presence indicator here instead).
    """
    if result["type"] == "silent":
        return "" # or e.g. a small
dot/indicator in a real UI
    elif result["type"] == "spoken":
        return result["message"]
    else:
        return "[unexpected response
format]"

if __name__ == "__main__":
    result = get_response("Just watching
the sunset, nothing really to say.")
    print(render(result) or "[silence]")

```

